

Chapter 2

Chapter 4

建立一个简单的数据通路

添加控制信号的数据通路

三种指令数据通路

加入jump指令后的数据通路

流水线

流水线数据通路

数据冒险：旁路与阻塞

控制冒险

异常

指令级并行

Chapter 5

局部性原理

存储器层次结构

命中和缺失

SRAM

DRAM

SDRAM

DDR

DIMM

闪存

磁盘

cache

虚拟存储器

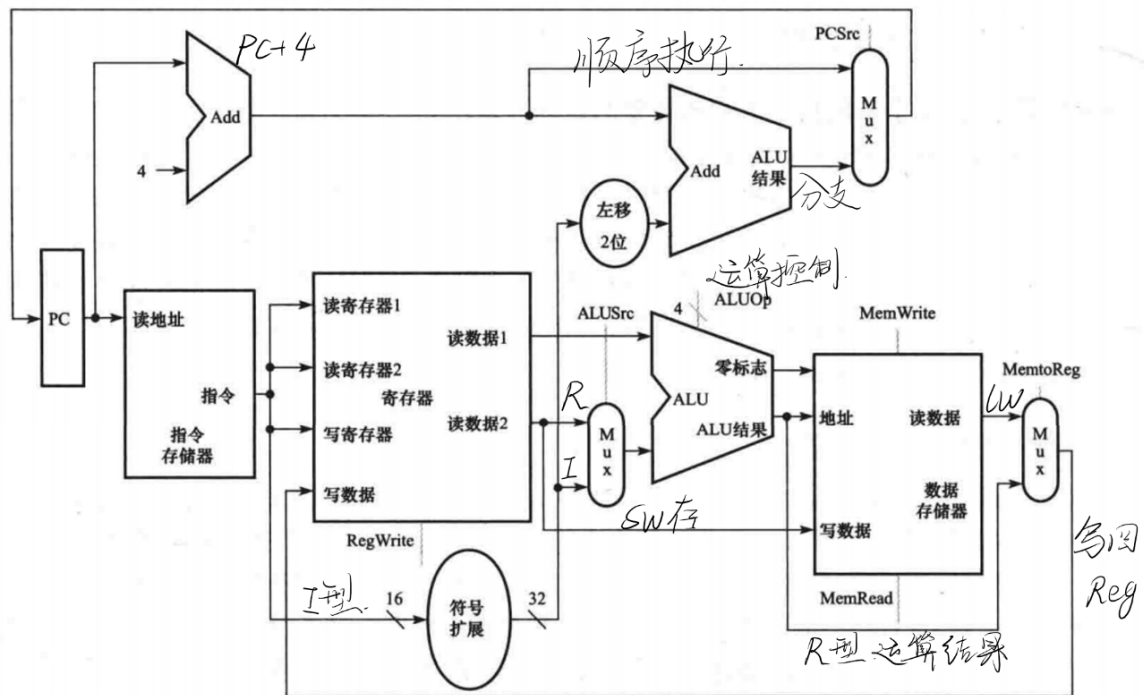
存储器层次结构

Chapter 2

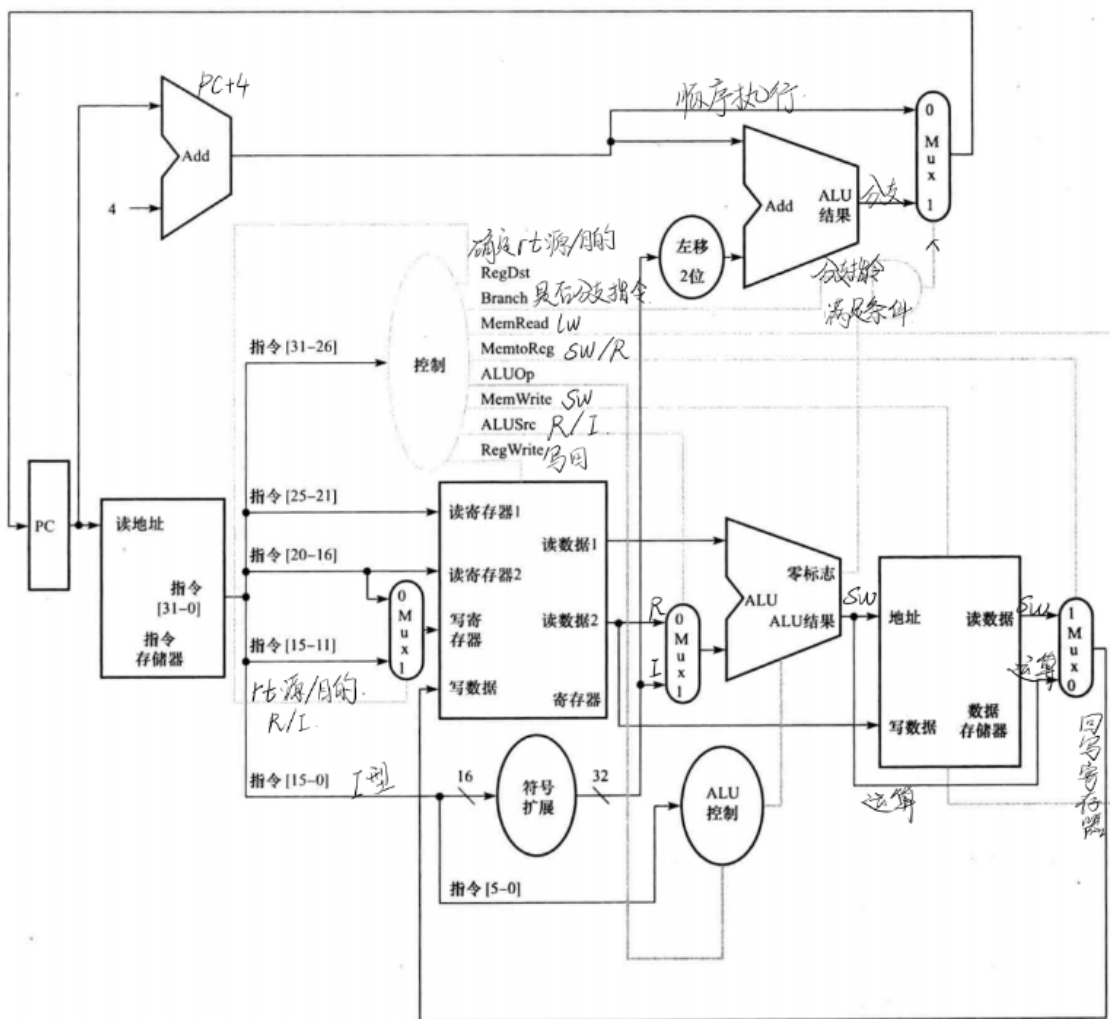
- 指令级设计的原则
 - 简单源于规整
 - 越小越快
 - 好的设计需要这种
 - 加速大概率事件
 - 最多只写入一个结果并且在流水线最后一级执行

Chapter 4

建立一个简单的数据通路



添加控制信号的数据通路

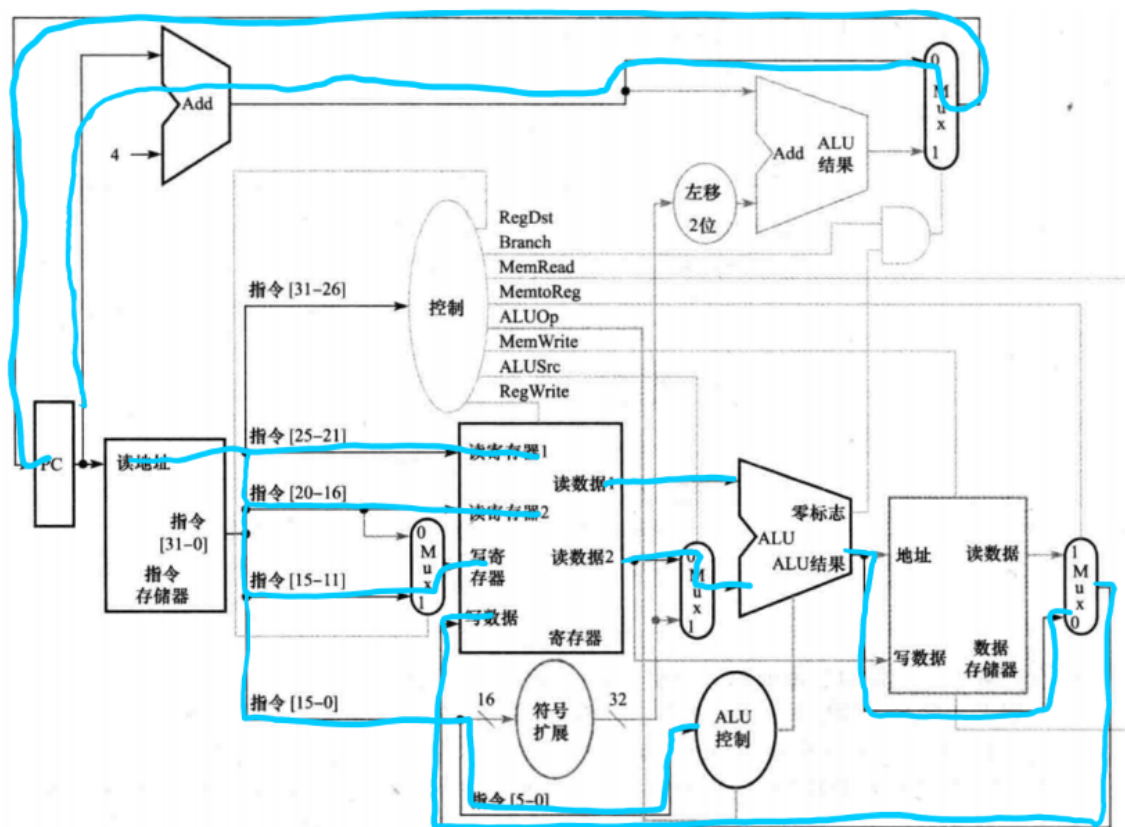


- 控制信号作用

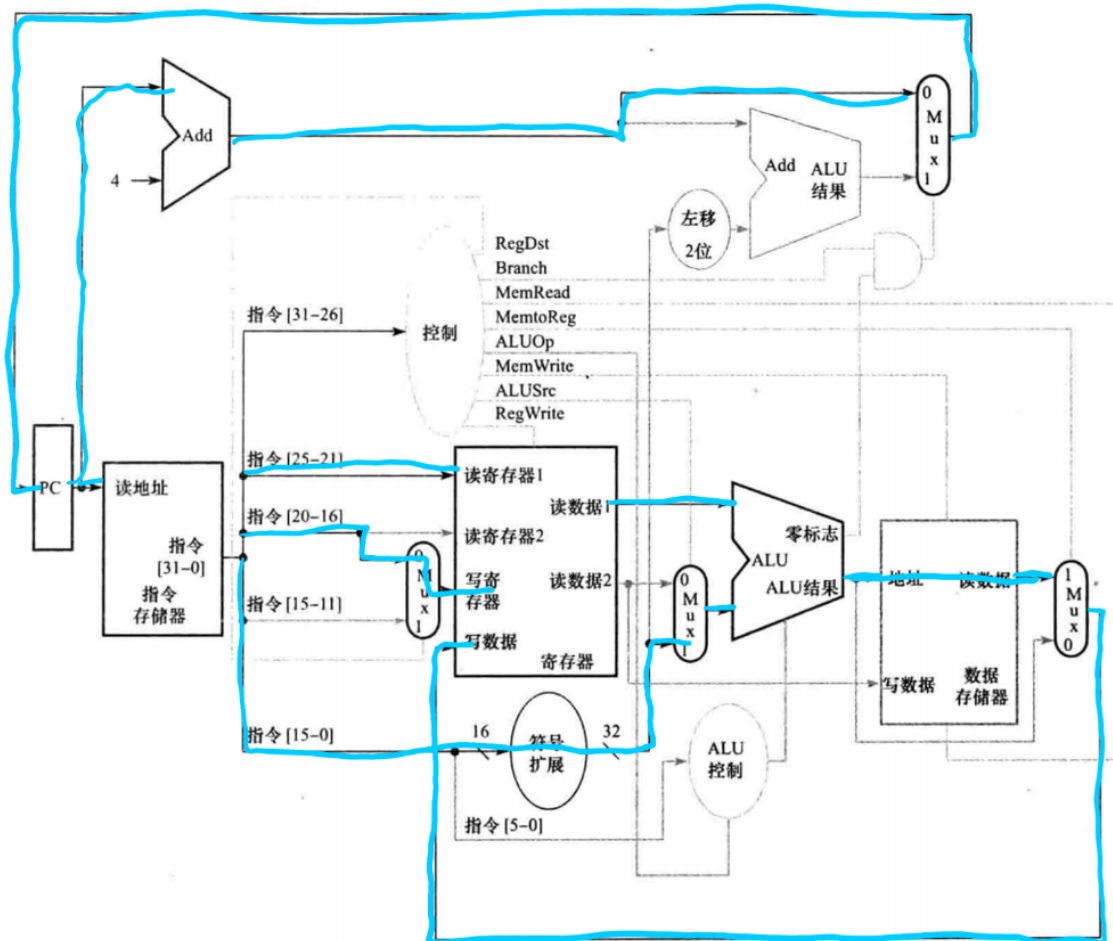
控制信号名	无效时的含义	有效时的含义
RegDst	写寄存器的目标寄存器号来自 rt 字段（位 20: 16）	写寄存器的目标寄存器号来自 rd 字段（位 15:11）
RegWrite	无	寄存器堆写使能有效
ALUSrc	第二个 ALU 操作数来自寄存器堆的第二个输出（读数据 2）	第二个 ALU 操作数为指令低 16 位的符号扩展
PCSrc	PC 由 PC + 4 取代	PC 由分支目标地址取代
MemRead	无	数据存储器读使能有效
MemWrite	无	将写入数据输入端的数据写入到用地址指定存储器单元中取
MemtoReg	写入寄存器的数据来自 ALU	写入寄存器的数据来自数据存储器

三种指令数据通路

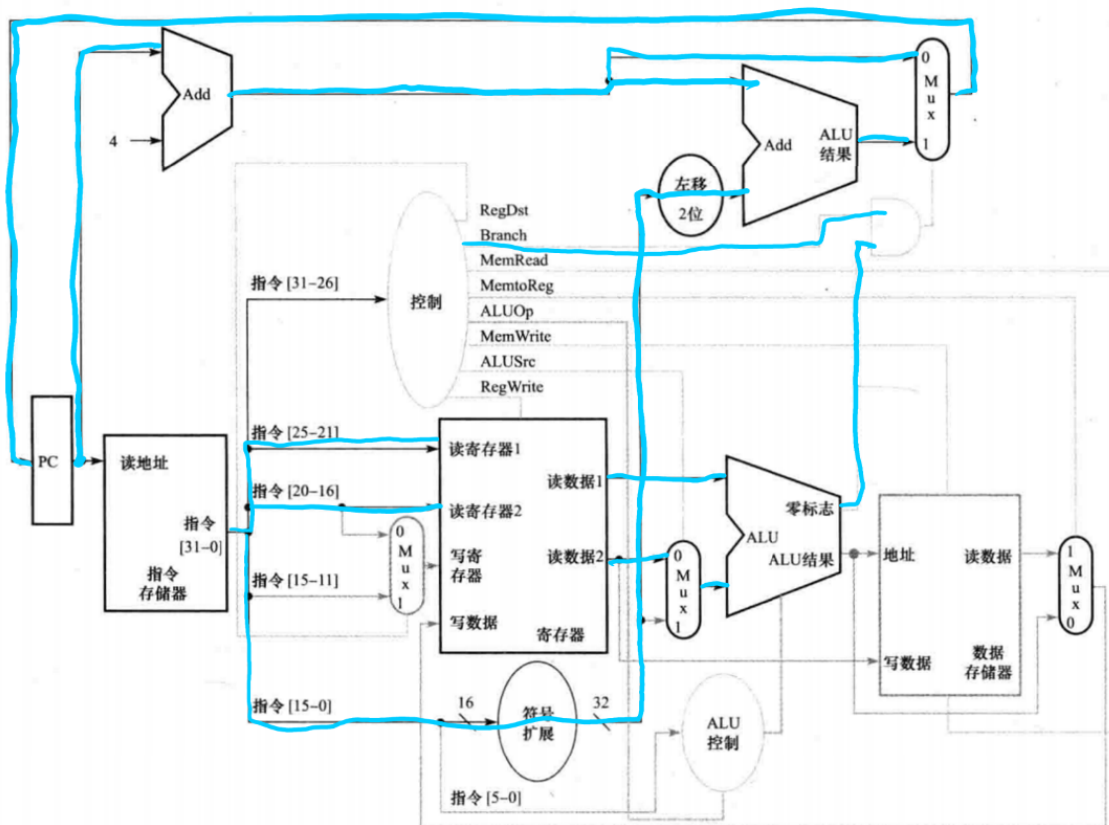
- R型算数指令



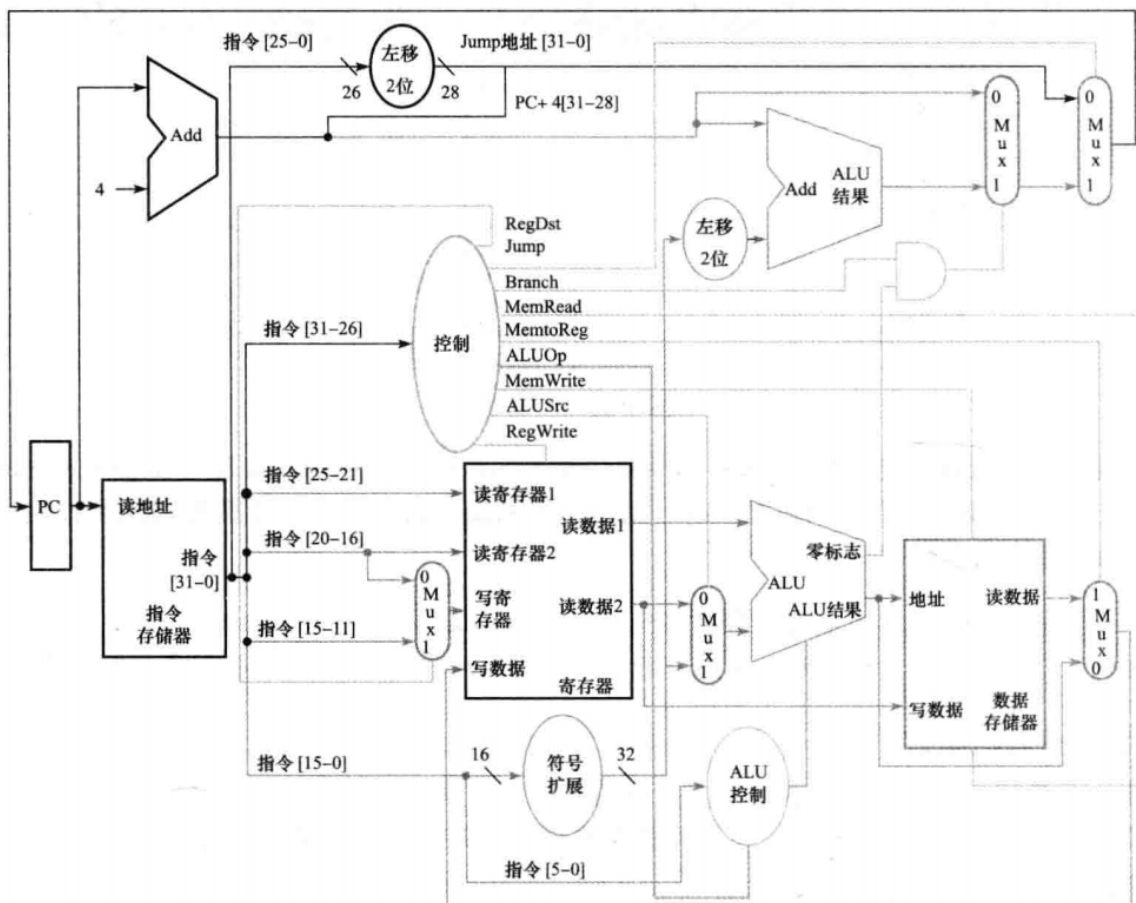
- 取数指令



- 相等则跳转指令



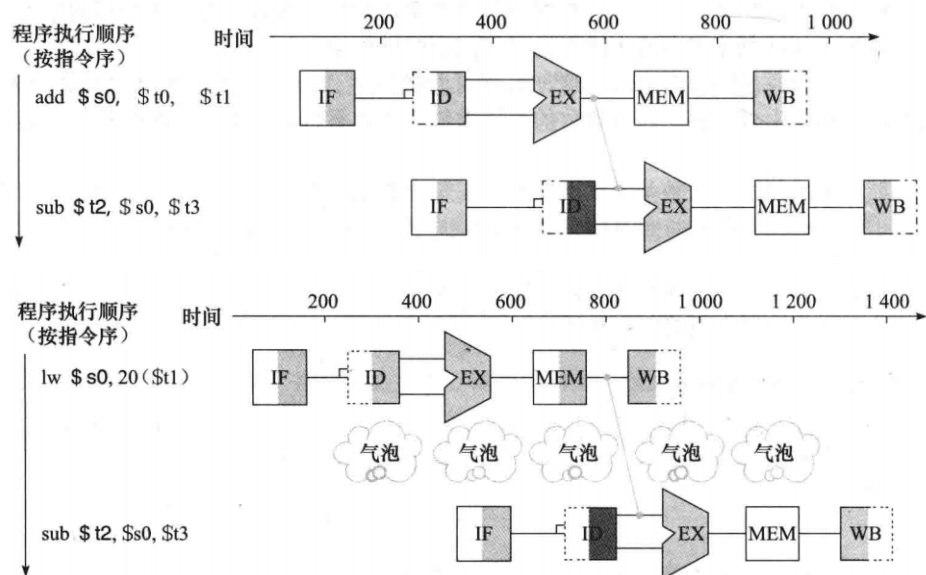
加入jump指令后的数据通路



流水线

- 单周期的缺点是，时钟周期对于所有指令等长，导致数据存取指令大大拉低时钟周期
- MIPS流水线的5个步骤
 - 从指令存储器中取出指令
 - 指令译码的同时读取寄存器
 - 执行算术操作或计算地址
 - 从数据存储器中取出操作数
 - 将数据写回寄存器
- 对寄存器堆的写操作发生在时钟周期的前半段，堆寄存器堆的读操作发生在时钟周期的后半段
- 流水线性能的提升是通过增加指令的吞吐率，而不是减少单条指令的执行时间来实现的
- 流水线冒险
 - 结构冒险：硬件缺乏相关支持导致指令不能同时访问一个硬件
 - 数据冒险：一条指令由于需要使用上一条指令的结果而发生的流水线暂停，可以通过forwarding或者bypassing解决

正常的数据冒险可以通过forwarding或者bypassing解决，而当出现**load-use**的时候，就必须增加一次流水线阻塞，也就是bubble



- 控制冒险：决策依赖于之前指令的结果，并且该指令正在执行

解决控制冒险的方法

■ 阻塞

由于不知道决策的结果，因此不能确定当前取到的指令是否应该执行

■ 预测

预测分支不发生是有意义的，因为即使分支发生了，也只需要浪费一个时钟周期，这与无论是否发生分支都进行阻塞的方式相比是相同的。

动态硬件预测器的预测取决于每一条分支的行为

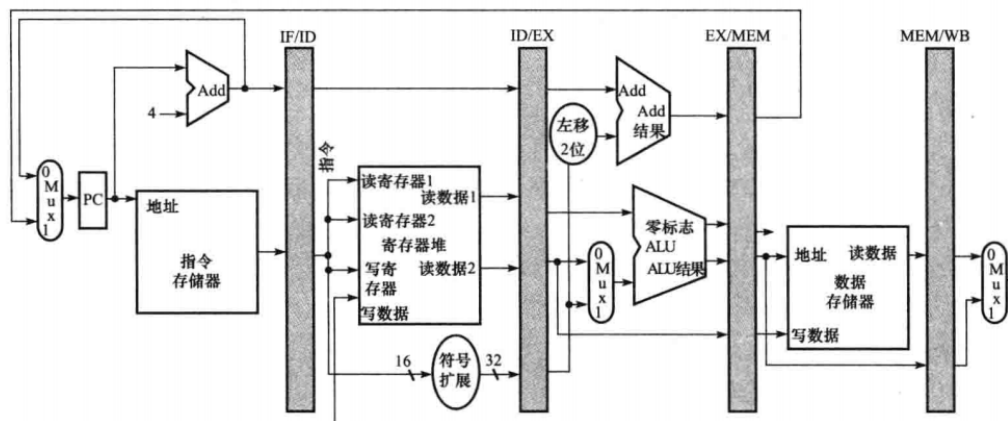
■ 延迟决定

延迟分支会顺序执行下一条指令，在一条指令执行完成之后再开始执行分支。MIPS编译器会在延迟分支指令之后紧跟一条不受分支影响的指令。

一般只用来处理一个时钟周期的延迟分支

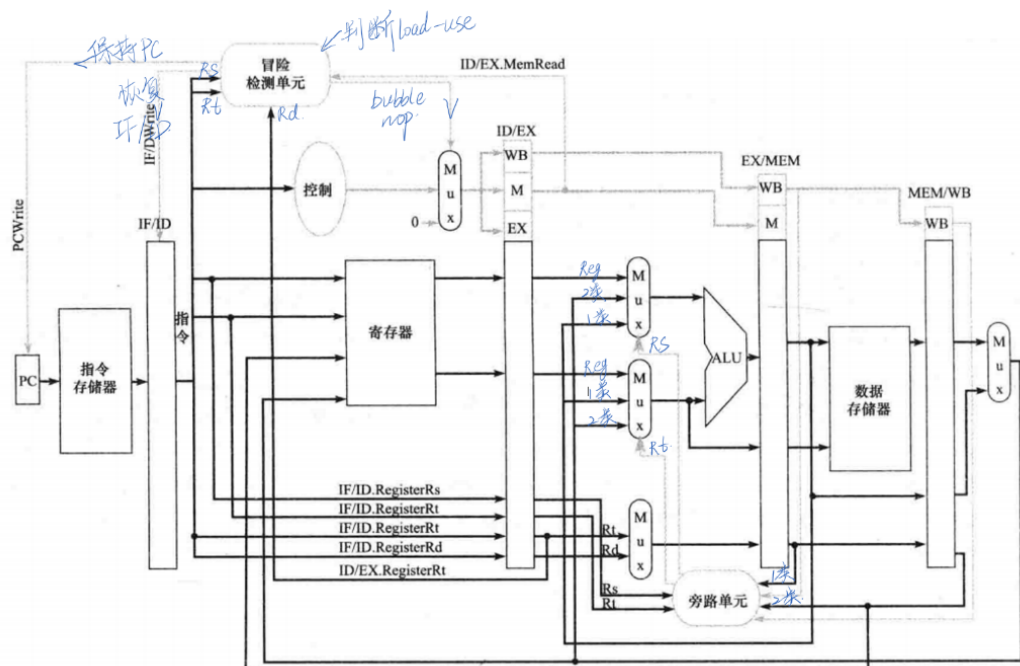
流水线数据通路

- 基本的流水线数据通路



- 上面的数据通路存在问题，在sw指令中，最终回写数据的时候，没有办法确定回写的寄存器号，因此需要额外在流水线寄存器中存储一个寄存器号

- MEM/WB.RegWrite
- MEM/WB.RegisterRd != 0
- MEM/WB.RegisterRd == ID/EX.RegisterRs || ID/EX.RegisterRt
- 和上一条不存在数据冒险，以下条件取反
 - EX/MEM.RegWrite
 - EX/MEM.RegisterRd != 0
 - EX/MEM.RegisterRd == ID/EX.RegisterRs || ID/EX.RegisterRt
- 旁路单元添加在EX级
- 数据冒险添加阻塞的条件
 - ID/EX.MemRead
 - ID/EX.RegisterRd == IF/ID.RegisterRs || IF/ID.RegisterRt
- 冒险检测单元添加在ID级
- 当处于ID级的指令被阻塞的时候，那么处于IF级的指令也要被阻塞。**防止这两条指令丢失的方法是保持PC寄存器和IF/ID流水线寄存器的值不变**
- 在流水线中插入空指令的方式就是ID/EX流水线寄存器的EX、MEM和WB级的控制信号都置为0
- 考虑数据冒险后的流水线数据通路



控制冒险

- 为了保证预取指令正确而导致的延迟叫做控制冒险或者分支冒险
- 解决方案
 - 假设分支不发生，继续顺序执行指令流

当分支发生跳转的时候，需要将流水线中的IF、ID和EX级的指令全部清除
 - 缩短分支的延迟

为了能够将分支决策提前，需要提前两个动作：计算分支目标地址和判断分支条件。

对于分支目标地址的计算：在IF/ID流水线寄存器中已经有了PC的值和立即数字段，因此可以将分支地址的计算从EX级提前到ID级，在指令解码的同时完成分支目标地址的计算。

对于判断分支条件：需要在ID级增加一个相等判断单元

- 为了解决数据冒险，还需要额外增加旁路单元，需要旁路的分支指令源操作数来自于EX/MEM或者MEM/WB。
- ID级进行比较分支比较所需要的数据可能在后面产生，因此有可能会发生数据冒险阻塞流水线。例如上一条是ALU指令，则必然需要发生阻塞。如果上一条是lw指令，则需要发生两个阻塞。

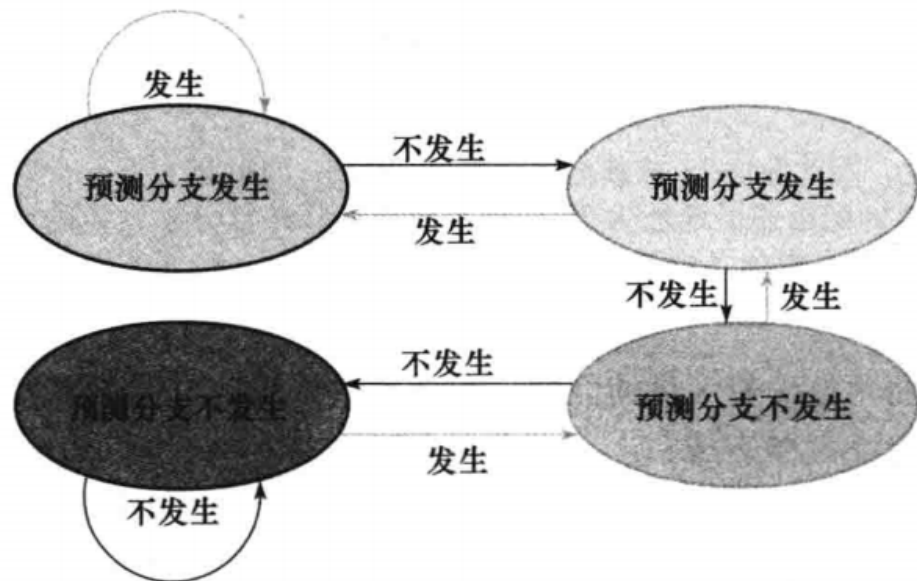
将分支执行提前到ID级是有意义的，因为他将分支预测错误的代价减小到了一条指令，就是分支执行时的正在取的那条指令。也就是在执行ID的时候，同时也在执行IF的那条指令

○ 动态分支预测

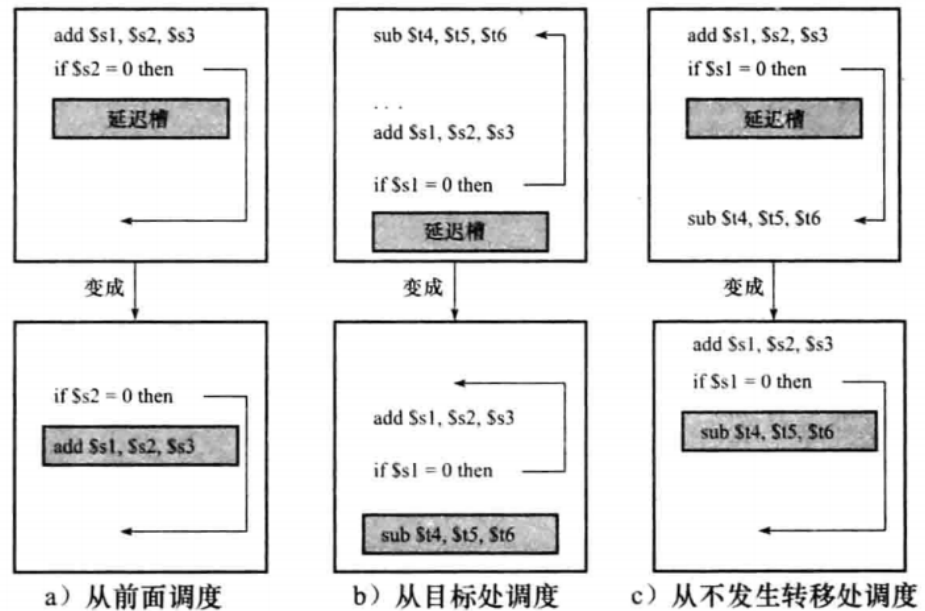
通过查找指令地址观察上一次执行该指令时分支是否发生，来决定目标分支

可以采用分支预测缓存或者分支历史记录表。分支预测缓存是一小块按照分支指令的地址低位索引的存储器区，其中包含一位或者多位数据用以说明分支最近是否发生过。

为了能够提高准确率，并不是在发生一次预测错误之后就反转状态信息，而是需要两次错误，也就是两位预测位的有限状态机。当分支经常发生或者经常不发生的时候，只会造成一次预测错误。



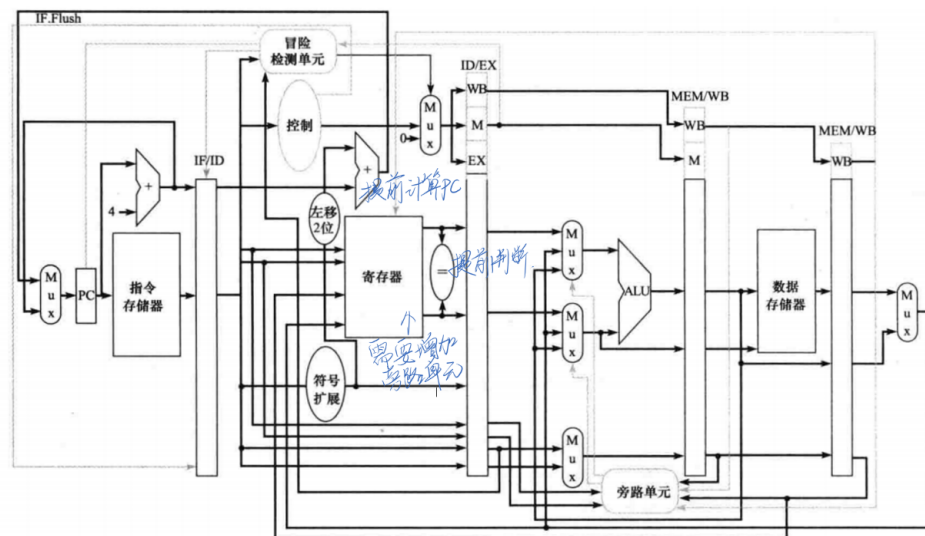
○ 延迟调度



○ 其他的预测手段

- 相关预测器：使用局部分支和最近执行分支的全局行为信息，能够产生更高的预测精度。
- 竞争预测器：对每个分支使用多个预测其，取最好的结果。目前精度最高。

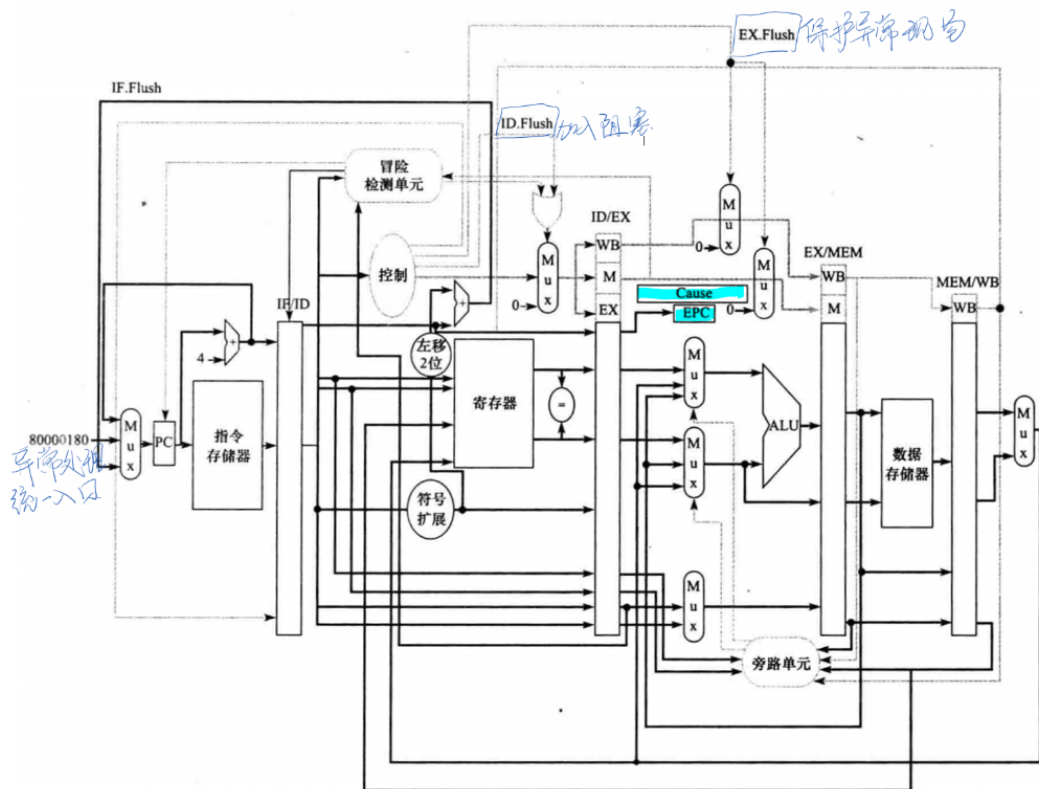
○ 考虑控制冒险后的流水线数据通路



异常

- 异常和中断：异常也叫做中断，指打断程序正常执行的内部事件，中断是来自处理器外部的异常
- 异常发生的时候处理器需要进行的基本操作是：
 - 在异常程序计数器(EPC)中保留出错指令的地址，并把控制权转移给操作系统的特定地址。
 - 为了处理异常，操作系统除了需要知道那条指令引起了异常，还需要直到引起异常的原因，主要有两种方法：
 - 设置一个状态寄存器(Cause)，其中有一个字段用于记录异常产生的原因

- 使用向量中断，控制权被转移到由异常原因决定的地址处。当异常不属于任何一个向量异常的时候，单个入口点供所有的异常使用。
- 增加ID.Flush和EX.Flush用来清除信号，实现阻塞
- 异常的溢出一般在EX级检测出来，可以用EX.Flush信号避免EX级的指令在WB级写回结果。
- 为了能够重新执行导致异常的指令，在异常处理的最后一步是将导致异常的指令地址保存在EPC中，由于保存的是PC+4，因此在重新执行的时候需要让PC=EPC-4。
- 加入异常处理的流水线数据通路



- 一个时钟周期内可能会发生多个异常，通常的解决方案是对异常按照优先级进行排序，一般都是按照异常发生的先后顺序。
- 精确中断和非精确中断
 - 精确中断：流水线计算机中的中断或者异常与导致中断或异常的指令精确相关
 - 非精确中断：流水线计算机中的中断或者异常不与导致中断或异常的指令精确相关

指令级并行

- 流水线挖掘了指令间潜在的并行性，有两种方法可以增加潜在的指令级并行程度
 - 增加流水线的深度以重叠更多的指令
 - 复制计算机内部部件的数量，使得每个流水级可以启动多条指令。多发射
- 高端微处理器尝试在每个时钟周期内发射3-6条指令，中端设计的目标是发射2条指令
- 多发射处理器一般有两种实现方式，静态多发和动态多发
 - 静态多发：决策在编译的时候由编译器完成
 - 动态多发：决策在执行的时候由硬件完成
- 多发射流水线中的两个问题
 - 往发射槽中发生的指令数量

- 静态多发在编译的时候由编译器决定
 - 动态多发在执行的时候由处理器决定，但编译器也会进行配合
- 处理数据冒险和控制冒险
 - 静态多发的部分甚至全部数据冒险和控制冒险都是由编译器静态处理
 - 动态多发通过硬件技术在执行的时候至少消除某些类型的冒险
- 推测：一种编译器或者处理器推测指令结果以消除执行其他指令对该结果的依赖的方法
 - 推测可以通过编译器或者硬件来完成。编译器可以通过推测对指令进行重排，处理器硬件在运行的时候同样可以完成这种交换
 - 推测错误的恢复机制
 - 对软件来说，编译器经常插入额外的指令检查推测的正确性并提供专门的修复例程。为了避免触发不必要的异常，加入额外的推测支持，使得这样的异常被暂时忽略，直到确认异常会发生
 - 对硬件来说，处理器一般缓存推测结果直至推测结果被确认。为了避免触发不必要的异常，通常将异常先暂时缓存。
- 静态多发发射处理器
 - 所有的静态多发发射处理器都使用编译器来帮助封装多条指令并处理冒险。
 - 一个时钟周期内发射多条指令，称为发射包，也叫做超长指令字
 - 绝大多数静态多发发射处理器也依赖编译器处理数据冒险和控制冒险，包括静态分支预测和代码调度，减少冒险或阻止冒险
 - 对下面这个循环进行双发射，并且采用循环展开

```

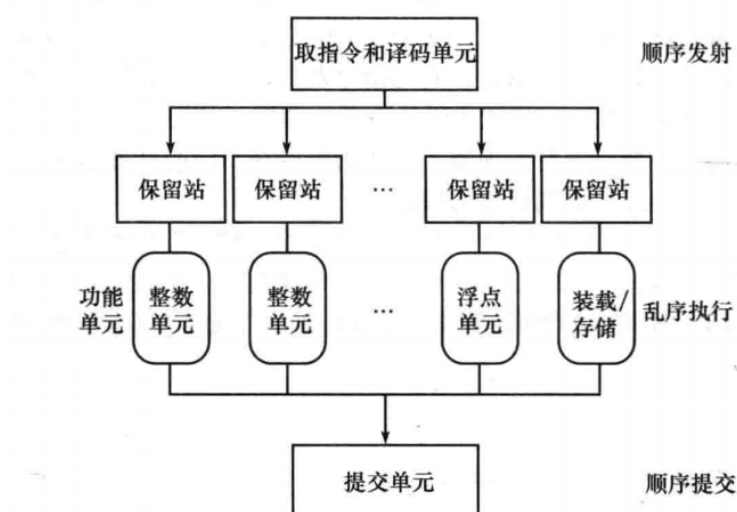
Loop: lw    $t0, 0($s1)    # $t0=array element
      addu  $t0,$t0,$s2# add scalar in $s2
      sw    $t0, 0($s1)# store result
      addi  $s1,$s1,-4# decrement pointer
      bne   $s1,$zero,Loop# branch $s1!=0

```

	ALU或分支指令	数据传输指令	时钟周期
Loop:	addi \$s1,\$s1,-16	lw \$t0, 0(\$s1)	1
		lw \$t1, 12(\$s1)	2
	addu \$t0,\$t0,\$s2	lw \$t2, 8(\$s1)	3
	addu \$t1,\$t1,\$s2	lw \$t3, 4(\$s1)	4
	addu \$t2,\$t2,\$s2	sw \$t0, 16(\$s1)	5
	addu \$t3,\$t3,\$s2	sw \$t1, 12(\$s1)	6
		sw \$t2, 8(\$s1)	7
	bne \$s1,\$zero,Loop	sw \$t3, 4(\$s1)	8

- 在循环展开中，需要进行寄存器重命名，消除虚假的数据依赖。
- 动态多发发射处理器
 - 动态多发发射处理器通常也称为超标量处理器，简称超标量，在超标量处理器中，不管代码是否经过调度，都是由硬件来保证执行的正确性的。
 - 动态流水线调度
 - 动态流水线调度选择下一条要执行的指令，可能的话会重排指令以避免阻塞。流水线被划分为3个重要单元：取指与发射单元、多个功能单元和提交单元。
 - 第一个单元取指并完成译码，将每条指令发送到对应的功能单元。

- 每个功能单元都有自己的缓冲区，称为保留站，用来保存操作数和操作。当缓冲区中包含了一条指令的操作和操作数之后，这条指令就能够被执行得到结果
- 得到结果后，被发送到等待该结果的保留站和提交单元
- 提交单元的缓冲区称为重排序缓冲区，可以用来提供操作数。



- 发射指令的时候被复制到相应的功能单元的保留站。如果操作数在寄存器堆中或者在重排序缓冲区中，那么这个操作数能够立刻被复制到保留站中，并且完成指令的操作，否则就会一直等待。当指令被发射之后，处理器所需要的寄存器就可以被覆盖了。
 - 如果操作数不在寄存器堆或者重排序缓冲区中，那么会是一个功能单元的计算结果，就需要类似于旁路的操作。
 - 动态流水线调度完成了：顺序发射-乱序执行-顺序提交的流程
 - 动态调度通常与基于硬件的推测机制相结合。对装在指令的目的地址进行预测，对存取指令进行重排序和利用提交单元避免错误的推测。
 - 为什么编译器不能够完成超标量处理器的动态调度
 - 并不是所有的阻塞都是可以事先知道的。例如cache缺失
 - 编译器不能够直到指令执行的准确顺序
 - 由于流水线的延迟和发射宽度不同，最佳的编译代码顺序也不是固定的
- 流水线和多发都提高了指令的吞吐率。

Chapter 5

局部性原理

- 时间局部性：最近访问过的之后还会访问
- 空间局部性：最近访问过的附近的之后会访问

存储器层次结构

- 一种由多存储器层次组成的结构，存储器的容量和访问时间随着离处理器的距离的增加而增加
- 两层之间进行信息交换的最小单元称为块

命中和缺失

- 命中率：在高层存储器中找到数据的存储访问比例
- 命中时间：访问存储器层次结构中的高层存储器所需要的时间，包括判断当前访问是否命中还是缺失的时间
- 缺失代价：将相应的块从底层存储器替换到高层存储器中，以及将该信息块传递给处理器所需要的时间之和

SRAM

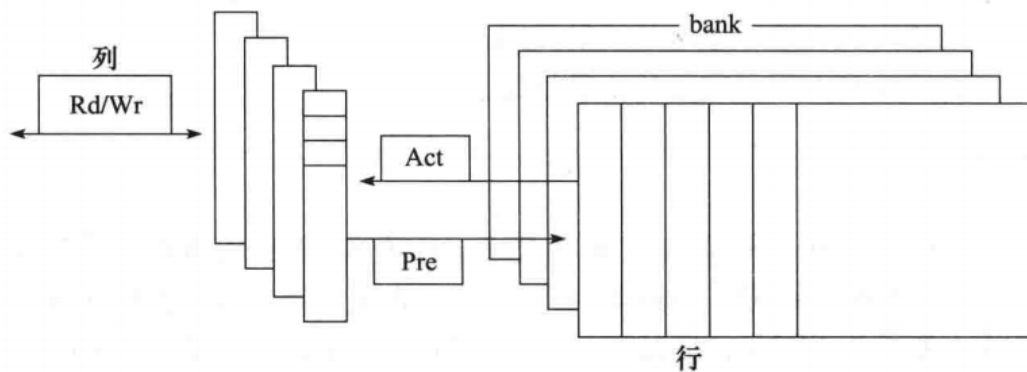
- 一种组织成存储阵列结构的简单集成电路，通常具有**一个读写端口**，虽然读写的时间不同，**但是SRAM对任何数据的访问时间是相同的。**
- SRAM不需要刷新，使用6-8个晶体管来存储

DRAM

- DRAM中，存储单元使用电容保持电荷的方式来存储数据。DRAM中存储每一位都只使用一个晶体管，因此比SRAM的密度高。
- DRAM上的电容保持电荷不能够持久保存数据，必须进行周期性刷新。

DRAM采取了一种两级译码的结构，可以通过在一个读周期后紧跟一个写周期来实现一次刷新。

当一行数据在缓冲器中时，无论DRAM的数据宽度是多少，都可以通过指定要传送的数据块大小和数据块在缓冲器中的起始地址的方式连续传送相邻地址的数据。



DRAM内部可以组织对多个bank进行读写操作，每个bank都有自己的一个缓冲器，向不同的bank发送同一个地址，可以允许同时对他们进行读写操作。这种访问方式称为地址交叉

SDRAM

- 为了进一步优化与处理器的接口，DRAM增加了时钟，称为同步DRAM，又是在于使用时钟对存储器的处理器保持同步。不需要额外指定地址位以突发的方式传送多个数据。

DDR

- 双数据速率SDRAM。可以在时钟的上升沿和下降沿都传输数据。

DIMM

- 双列存储器模块
- 通常含有4-16块DRAM芯片，组织成8位位宽。
- 使用rank来表示DIMM中的芯片的一个子集

闪存

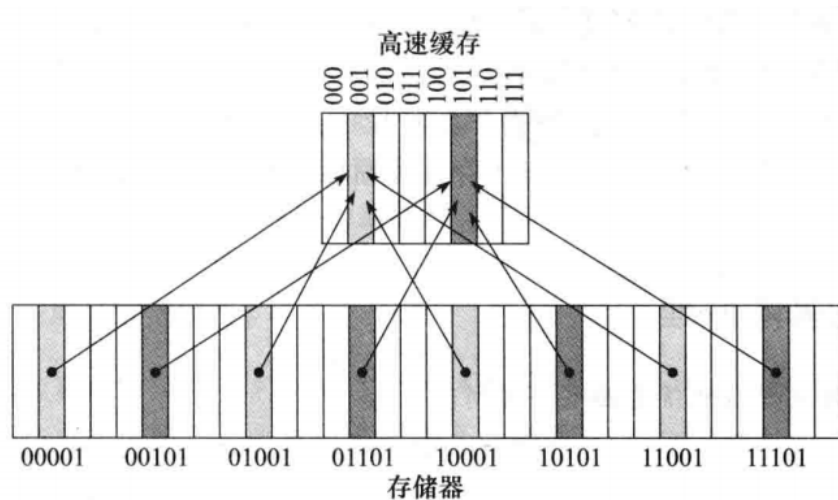
- 一种电可擦除的可编程只读存储器（EEPROM）
- 对闪存的读写操作可以使存储位损耗，因此需要使用损耗均衡的技术来提高使用寿命。

磁盘

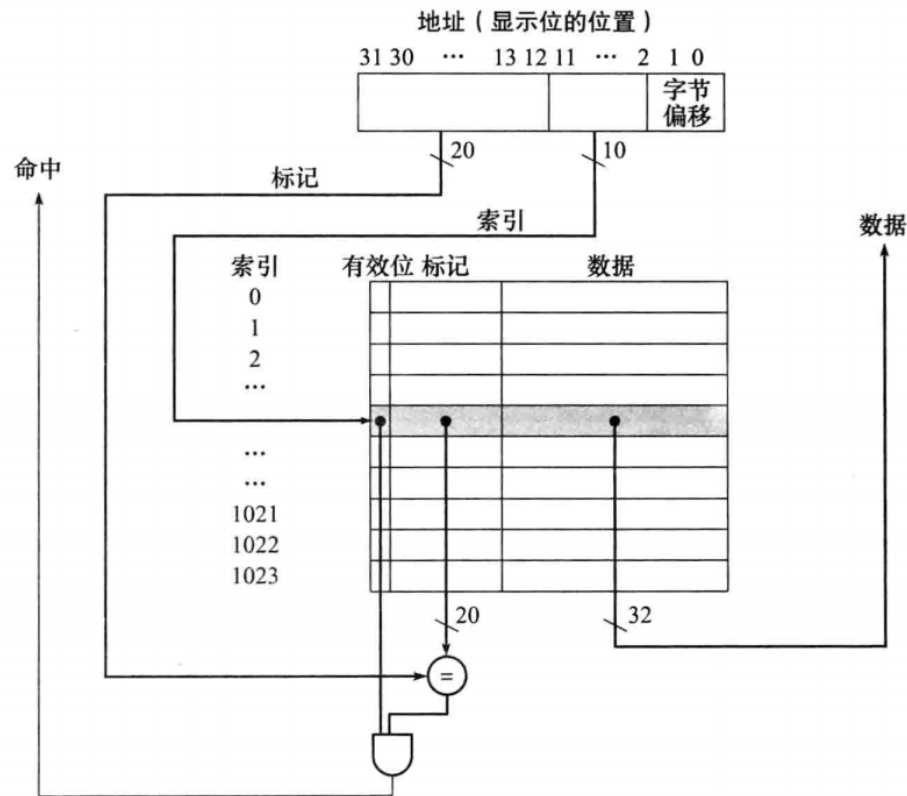
- 磁道、扇区、柱面
- 寻道时间、旋转延时、传输时间
- 磁盘不像随机访问设备
- 为了获得更好的性能，应该尽可能使得所有的扇区的数据密度相同，因此顺序的块可能在不同的磁道上

cache

- 最简单的cache结构设计包含索引，标记和有效位
 - 索引：根据cache块的大小，将主存通过模运算映射到cache中



- 标记：索引是低位，标记就取高位，这样就能够直接获得主存地址
- 有效位：判断某一个索引是否用过



• cache大小的计算

- cache中有 2^n 个块，需要 n 位来索引
- 每个块的大小为 2^m 个字，对应 2^{m+2} 个字节，需要 m 位来标记
- 需要有效位1位
- 标记域大小 $32 - (m + n + 2)$

cache的总大小 = cache中的块数 × (块大小 + 标记域大小 + 有效位)

$$size = 2^n \times (2^{m+2+3} + 32 - (m + n + 2) + 1)$$

一般讨论cache的大小只考虑数据的大小

• 增加cache大小带来的影响

- 增加块大小通常会引起缺失率的降低，但是当块大小增加到一定程度的时候，缺失率就会上升
- cache中的块的数量减少，块之间的竞争变大
- 块中各个字之间的空间局部性降低
- 缺失成本上升，将一个块从低层存储传输到高层存储的时间变长
 - 提前重启：当块中所需要的字返回的时候，就马上继续执行，不需要等到整个块都传输完成
 - 请求字优先/关键字优先：被请求的字先从存储器传输到cache中，然后再传输块中的其他数据

• cache缺失

- cache发生缺失的时候，需要两部分共同完成
 - 处理器控制单元
 - cache缺失必然引起流水线的阻塞，cache缺失的时候，需要等待主存操作完成。
 - 进行初始化主存访问和重新填充cache的独立控制器

- 指令缺失（数据缺失类似）
 - 把PC的值-4还原，并送到存储器中
 - 通知主存执行一次读操作，并等待主存访问完成
 - 写cache，将主存取回的数据写入cache中存放数据的部分，然后将地址的高位写入标记域，设置有效位
 - 重启指令，重新取指
- 写操作
 - 一般CPU的写操作只将数据写入cache，这就会导致支持cache和主存的不一致。可以使用以下方法来保持主存和cache的一致性
 - 写直达：写操作总是同时更新cache和下一存储器层次，但这样会极大影响性能
 - 写缓冲：当数据等待被写入主存的时候，会先写入写缓冲中。当数据写入cache和写缓冲后，处理器就可以继续执行。当写主存操作完成后，写缓冲中的数据就可以被释放了。如果写缓冲已经满了，就必须等待写主存操作完成空出一个位置来。
 - 写回法：当写操作发生的时候，新值仅仅写入cache中，只有当修改过的块被替换的时候，才需要写回低层存储结构中。不需要替换的时候，都是从cache中直接访问的，即使有不一致也不影响
- cache性能评估
 - CPU时间

$$CPU\text{时间} = (CPU\text{执行时钟周期数} + \text{存储器阻塞时钟周期数}) \times \text{时钟周期}$$
 - 存储器阻塞时钟周期数

$$\text{阻塞时钟周期数} = \text{读操作阻塞时钟周期数} + \text{写操作阻塞时钟周期数}$$

$$\text{读操作阻塞时钟周期数} = (\text{读次数}/\text{程序数}) \times \text{读缺失率} \times \text{读缺失代价}$$

$$\text{写操作阻塞时钟周期数} = (\text{写次数}/\text{程序数}) \times \text{写缺失率} \times \text{写缺失代价} + \text{写缓冲阻塞}$$

当不考虑缓冲区阻塞代价的时候

$$\text{阻塞时钟周期数} = (\text{指令数}/\text{程序数}) \times (\text{缺失数}/\text{指令数}) \times \text{缺失代价}$$
 - 平均存储器访问时间 AMAT

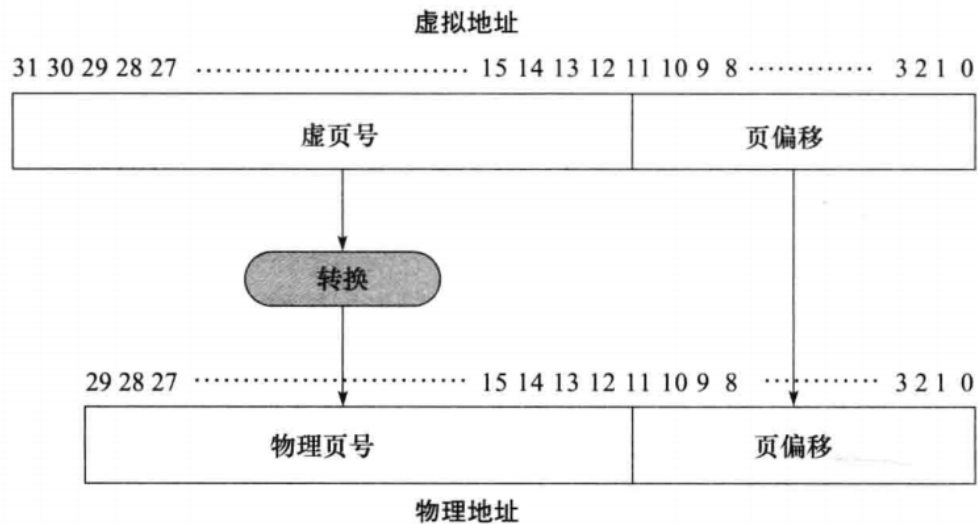
$$AMAT = \text{命中时间} + \text{缺失率} \times \text{缺失代价}$$
- cache改进
 -

虚拟存储器

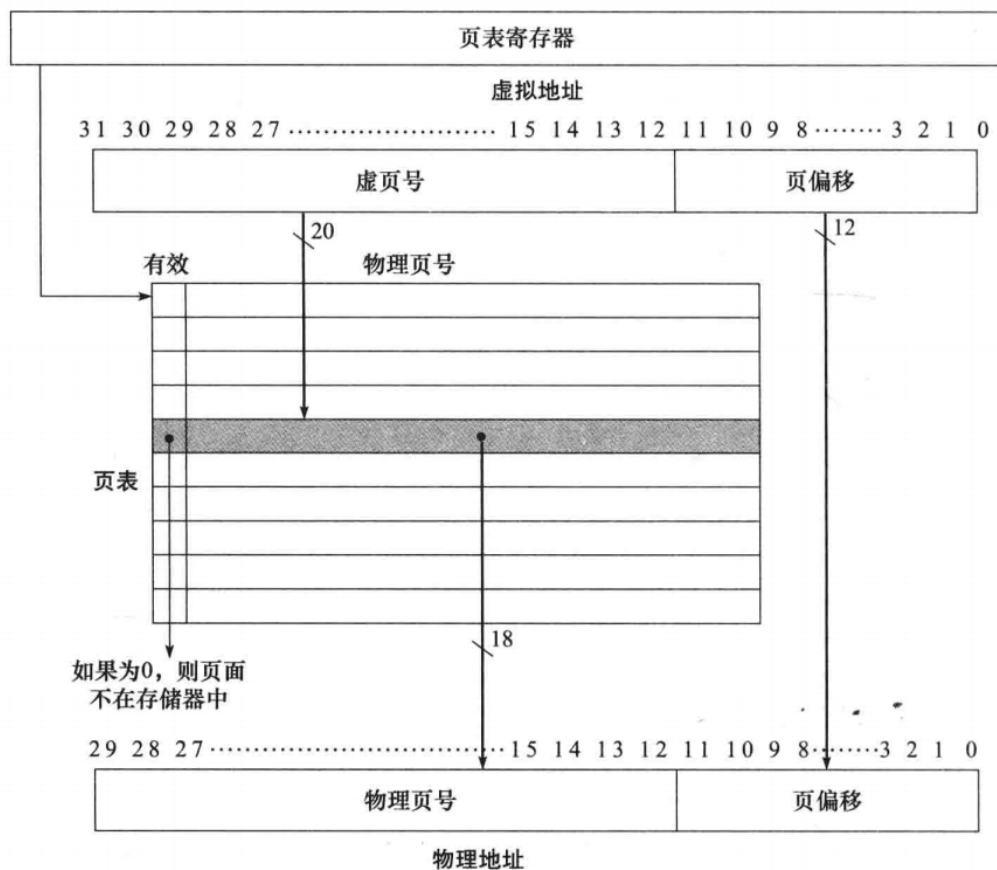
- 虚拟存储器是一种将主存用作辅助存储器的高级缓存技术
- 两大动机
 - 实现了程序地址空间到物理地址的转化，加强了各个程序地址空间的保护
 - 允许单用户程序使用超过主要存储器的容量
- 虚拟存储器中，块被称为页，访问缺失被称为缺页。处理器产生一个虚拟地址，结合软硬件转化成一个物理地址的过程称为地址映射或地址转换。一个虚页可能被映射到主存中的物理地址也可能被映射到磁盘上。物理页可以被两个指向相同物理地址的虚拟地址共享，因此可以在多个程序之间共享数据和代码
- 重定位：将程序所需要的虚拟地址映射到不同的物理地址

重定位的方法允许我们将程序装载到主存的任何位置

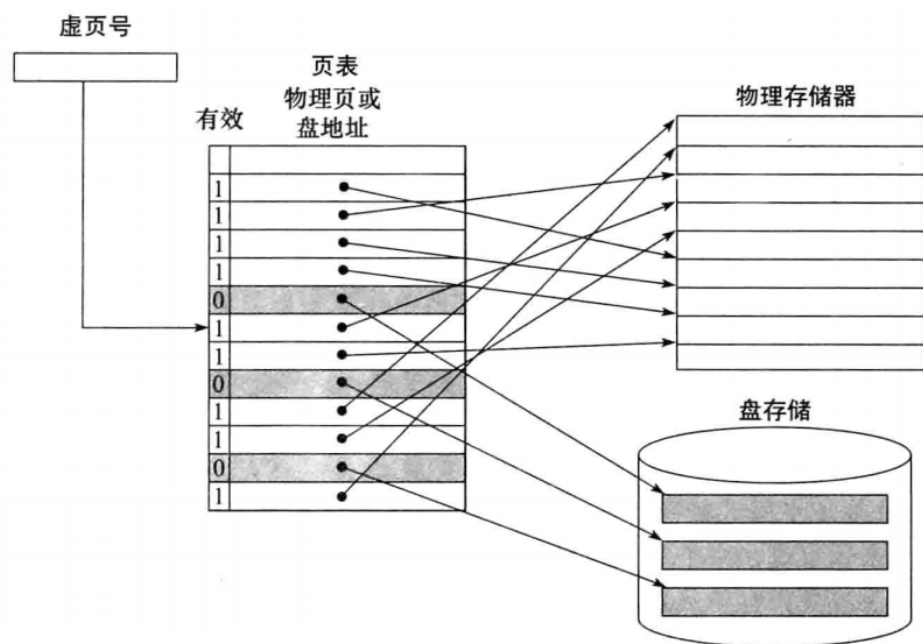
- 虚拟地址到物理地址的映射关系



- 在虚拟存储器中，地址被划分成了虚页号和页偏移
 - 虚页号通过一个转化映射到物理页号，页偏移之间补在后面
 - 页偏移决定了地址大小
 - 虚拟地址的可寻址空间可以大于物理地址的可寻址空间
- 虚拟存储器设计的关键性因素
 - 为了弥补较长的访问时间，页应该足够大
 - 能降低页缺失率的组织架构具有吸引力
 - 缺页可以用软件来处理，相较于磁盘访问时间，软件处理开销更小
 - 由于写时间太长，因此在虚存中不采用写直达，而是采用写回机制
- 页表
 - 在虚拟存储器中，使用一个索引存储器的表来定位页。
 - 页表存放在存储器中
 - 页表使用虚拟地址中的页号作为索引，已找到相应的物理页号
 - 每个程序都有自己的页表
 - 硬件包含一个指向页表首地址的寄存器——页表寄存器



- 有效位开启，页表提供虚页对应的物理页号，有效位关闭，页表提供虚页对应的磁盘地址

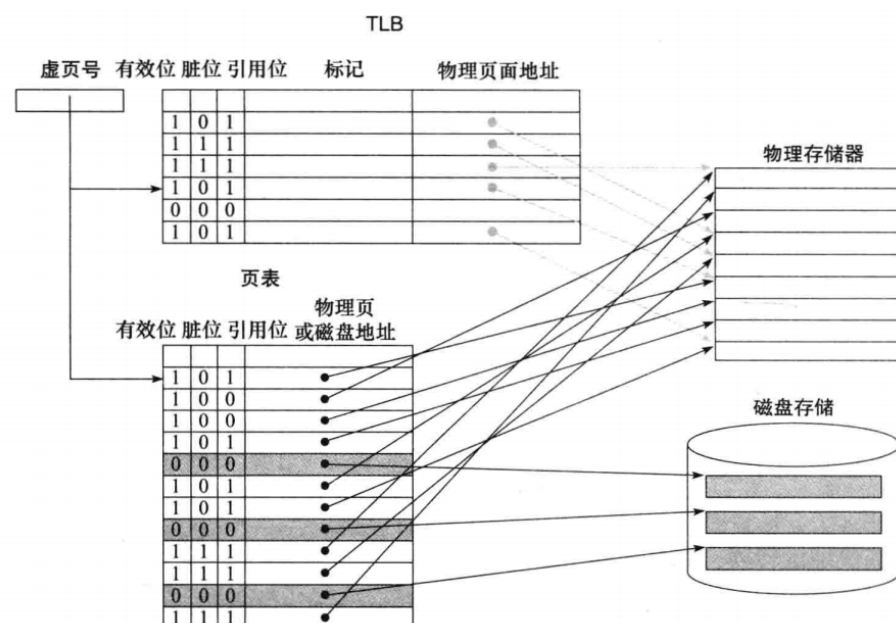


- 交换区：为进程的全部虚拟地址空间所预留的磁盘空间，为了方便将存储器中的某一页替换出去，同时创建记录每个虚拟也在磁盘上的存储位置。当主存满的时候，就需要替换出一个页，将这个替换出去的页写入交换区
- 虚拟内存的写操作：写直达法会直接对磁盘进行读写操作，这个时钟周期太长。因此采用写回机制，对存储器中的页进行单独的邪恶操作，并且在该页被替换出存储器的时候才会复制到磁盘中

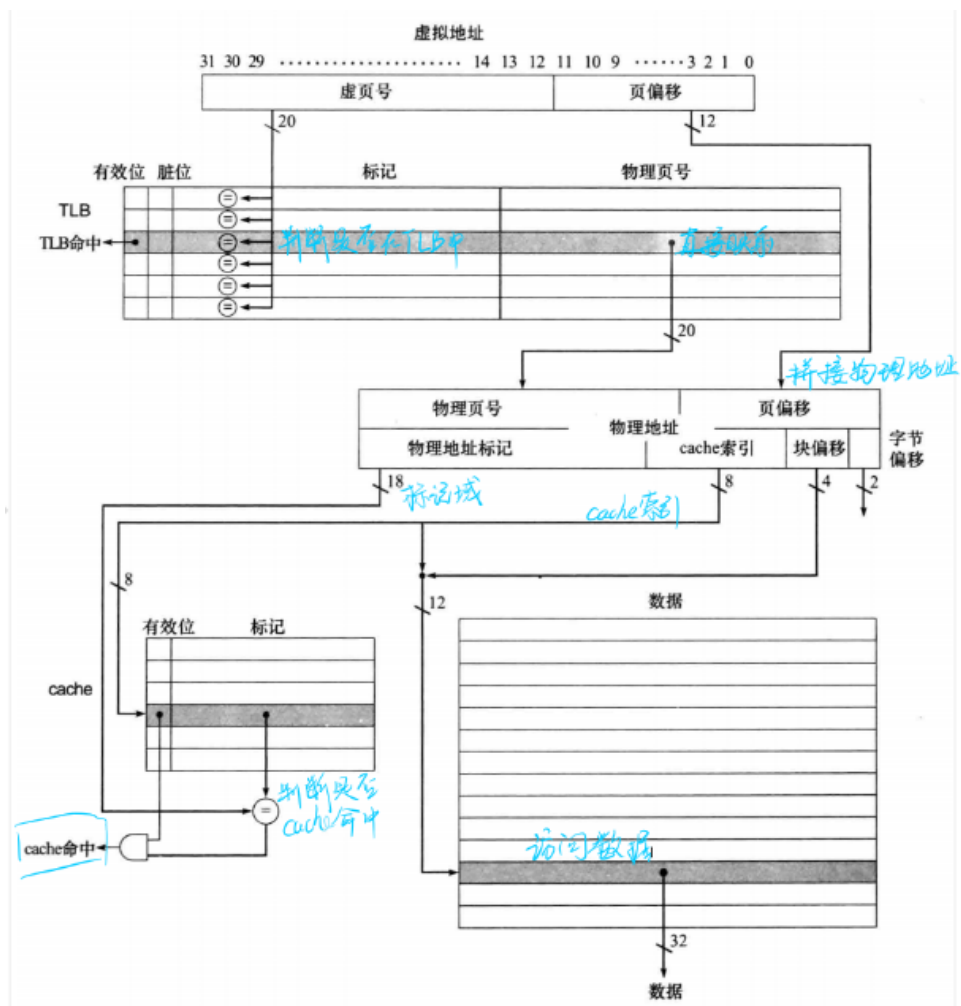
相对于磁盘访问时间，传输时间要少的多，因此把整页复制到磁盘中比把单个字节写回要高效，但是开销很大，因此加入脏位来判断是否需要写回磁盘，对于修改过的页设置脏位，写回磁盘

- TLB

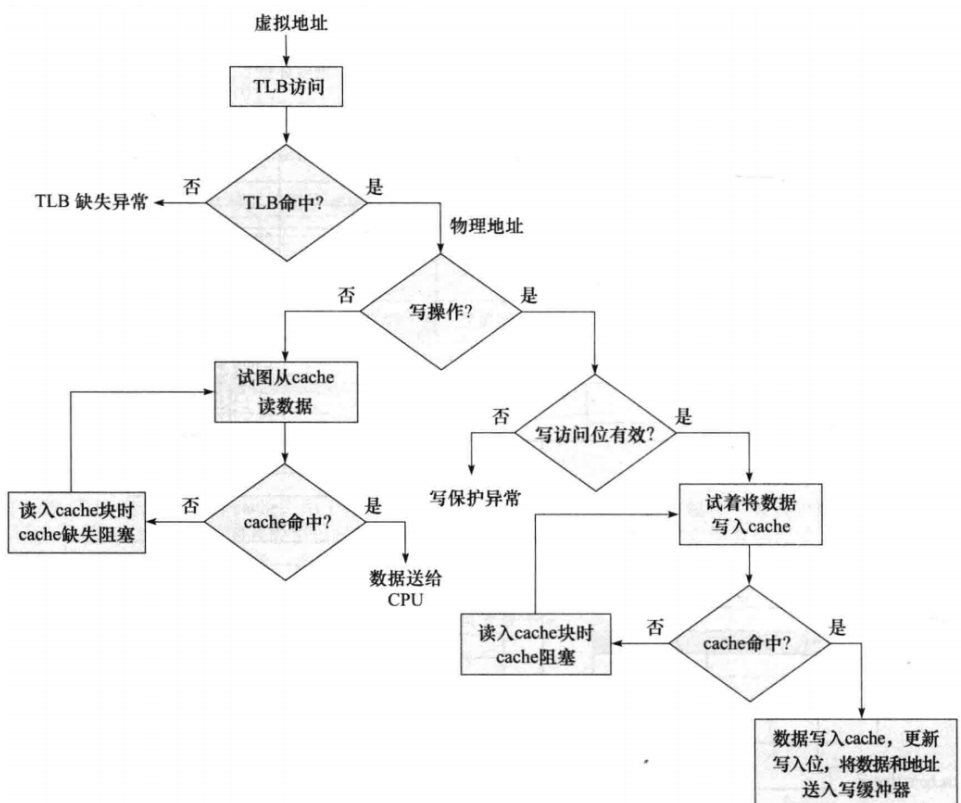
- 由于页表存放在主存中，因此程序每次访存至少需要两次，先访问页表取得物理地址，然后再访存取得数据
- 现代**处理器**使用了一个特殊的cache以追踪最近使用过的地址变换。称作快表 TLB，用于记录最近使用地址的映射信息的高速缓存
- TLB每个标记项存放虚页号的一部分，每个数据项中存放物理页号，其他还包括脏位和引用位
- 每次访存时
 - 都先在TLB中查询是否命中，如果命中就可以直接获得物理页号形成地址，并将引用位标记。如果是写操作还需要标记脏位。
 - 如果TLB中没有命中，则需要判断是缺页还是TLB缺失。
 - 如果页在主存中，则只是发生了一次转换缺失，需要通过页表中的变换装载到TLB中并重新访问
 - 如果页不在主存中，则发生了缺页，需要调用操作系统来处理异常。
 - TLB中的项数比主存要少很多，因此发生TLB缺失比发生缺页频繁
 - 发生TLB缺失的时候，选哟在页表中找到缺失对应的地址转换，并从TLB中选择一项进行替换，同时将引用位和脏位写到页表中的对应位置。TLB中只能修改引用位和脏位，对其的修改采用写回机制——只有当TLB中的项被替换出去的时候才将引用位和脏位写入页表。



- 由于TLB中的项数不多，所以一般采用全相联的方式，因为全相联具有较低的缺失率，并且由于项数少，其映射成本也不高。
- 一个TLB和cache串联的例子



整体的流程如下



- 虚存、TLB和cache的集成

- 操作系统决定将某一个页移到磁盘上的时候, 需要先将该页中在cache上的数据刷新到内存中, 然后修改页表和TLB, 将这个页删除, 最后写到磁盘上。

- 在一次存储器访问的过程中可能遇到三种不同类型的缺失：TLB缺失、缺页、cache缺失，考虑不同缺失的组合

TLB	页表	cache	这种情况可能发生么？如果可能，在什么情况下发生？
命中	命中	缺失	可能，但若 TLB 命中就不可能检查页表
缺失	命中	命中	TLB 缺失，但在页表中找到表项；重试后在 cache 中找到数据
缺失	命中	缺失	TLB 缺失，但在页表中找到表项；重试后在 cache 中未找到数据
缺失	缺失	缺失	TLB 缺失，随后发生缺页；重试后在 cache 中必找不到数据
命中	缺失	缺失	不可能：如果页不在主存中，TLB 中没有此转换
命中	缺失	命中	不可能：如果页不在主存中，TLB 中没有此转换
缺失	缺失	命中	不可能：如果页不在主存中，数据不允许在 cache 中存在

注意到只要页表缺失，cache就不可能命中，这是因为页表缺失表示对应的数据块没有装载到内存，因此更不可能在cache中。TLB命中了页表就不可能缺失，而且TLB命中了就不会再访问页表

- 物理cache和虚拟cache
 - 物理cache：使用物理地址作为cache索引和标记，每次访存都需要经过TLB
 - 虚拟cache：使用虚拟地址作为cache索引和标记，访存的时候不需要经过TLB
- 虚存的保护
 - 多个进程共享一个主存，任何一个进程都不能修改另一个进程的地址空间。TLB中的写访问为就可以防止一个页被改写。
 - 写访问为可以用来把共享限制为只读，并且和页表中其他位一样，该位只能被操作系统修改。当p1想要访问p2中的某个也的时候，p2就要请求**操作系统**在p1的地址空间中为虚拟页生成一个页表项，指向p2共享的物理页。并且可以根据权限设置写保护位。写保护位不仅要在页表中还要在TLB中
 - 为了方便上下文的切换（进程的切换），增加了进程标识符来区分不同的进程，只有当进程标识符以及TLB标记部分都匹配的时候才能证明数据是所需要的，这样可以避免不必要的对TLB的清空
- TLB缺失和缺页
 - 当TLB中没有一个表项可以匹配虚拟地址的时候，就会发生TLB缺失，有以下两种情况
 - 页在主存中，只需要通过页表创建缺失的TLB项
 - 页不在主存中，要通过异常的范式将控制权交给操作系统来解决缺页
 - 缺页将在主存访问时钟周期的某一时刻被发现，为了在缺页处理完成后能够重新启动引起缺页的指令，必须保存该指令的程序计数器中的值到EPC中
 - TLB缺失或者缺页异常必须在访存发生的同一个时钟周期的末尾被判断，因此下一个时钟周期就可以开始进行异常处理而不是正常进行取指操作
 - 操作系统知道了引起缺页的虚拟地址之后
 - 使用虚拟地址查找页表项，并在磁盘上找到被访问的页的位置
 - 选择替换一个物理页，如果被需要的页被修改过，需要将这个物理页写回到磁盘上
 - 启动读操作，将被访问的页从磁盘上取回到所选择物理页的位置上
 - 可重启指令：一种在异常被处理之后能够从异常中恢复而不会影响到指令的执行结果的指令

- TLB缺失处理程序并不检查页表项是否有效，所以操作系统对页表中的页表项并不做检查就直接装载到TLB并重新执行指令。如果表项无效，另一个异常（缺页）就发生。这种方法让频繁发生的TLB缺失处理耕畜但却影响了缺页处理的性能
- 采用以下方式来降低缺失率
 - 增大页的容量，以充分利用空间局部性降低缺失率
 - 使用页表实现的虚拟地址和物理地址之间的映射采用全相联的方式，这样虚拟页就可以被放置到主存的任何位置
 - 操作系统使用类似LRU和访问为之类的技术来选择替换哪一页
- 抖动：如果一个程序经常访问比它拥有的物理存储器多的虚拟存储器，则会不断在存储器和磁盘之间交换页面

存储器层次结构
